Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Chapter 9
CSparse

This chapter provides an overview of the use of CSparse as a stand-alone sparse matrix package available for download from www.siam.org/books/fa02. CSparse functions are divided into three sets: primary, secondary, and tertiary. The primary functions are all that an application needs to create a matrix A, solve $Ax = b$, and perform basic matrix operations. Secondary routines include the ordering and factorization methods, forward/backsolve, rank-1 update/downdate, and routines for manipulating permutation vectors. These can be useful in some applications (when a matrix is analyzed and ordered once but factorized multiple times with different numerical values but identical nonzero pattern, for example). Tertiary routines would rarely be used in any application; they are primarily meant as functions for use within CSparse itself.

Every file in CSparse starts with the statement `#include "cs.h"`, as must every program that uses CSparse. It defines the `cs` sparse matrix data structure and the prototypes for each function. The `cs.h` file is split into three sections (primary, secondary, and tertiary routines). The routines in each set are described below. Each parameter or return value of CSparse is described as one of the following:

- **in**: The parameter must be defined on input. It is not modified.
- **in/out**: The parameter must be defined on input. It is modified on output.
- **out**: The memory space for the parameter must be allocated on input. Its contents are not defined on input. It is modified and defined on output.
- **work**: The memory space for the parameter must be allocated on input. Its contents are typically not defined on input or output.
- **returns**: The return value of each CSparse function. CSparse functions that return a pointer return `NULL` if an error occurs.

If not otherwise specified, `m` is the number of rows in the sparse matrix, `n` is the number of columns, and all CSparse routines expect matrices in compressed-column form. The `order` parameter is used in several routines: `0` results in the

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

natural ordering; `1` is a minimum degree ordering of $A + A^T$; `2` is a minimum degree ordering of $S^T S$ where $S = A$, except rows with more than $10\sqrt{n}$ entries are removed; and `3` is a minimum degree ordering of $A^T A$.

9.1 Primary CSparse routines and definitions

`cs`: sparse matrix in compressed-column or triplet form

```
typedef struct cs_sparse    /* matrix in compressed-column or triplet form */
{
    int nzmax ;     /* maximum number of entries */
    int m ;         /* number of rows */
    int n ;         /* number of columns */
    int *p ;        /* column pointers (size n+1) or col indices (size nzmax) */
    int *i ;        /* row indices, size nzmax */
    double *x ;     /* numerical values, size nzmax */
    int nz ;        /* # of entries in triplet matrix, -1 for compressed-col */
} cs ;
```

`cs_add`: $C = \alpha A + \beta B$

```
cs *cs_add (const cs *A, const cs *B, double alpha, double beta) ;
```

Adds two sparse matrices, $C = \alpha A + \beta B$.

`A`	in	sparse matrix
`B`	in	sparse matrix
`alpha`	in	scalar
`beta`	in	scalar
	returns	`C=alpha*A+beta*B`; `NULL` on error

`cs_cholsol`: solve $Ax = b$ using Cholesky factorization

```
int cs_cholsol (int order, const cs *A, double *b) ;
```

Solves $Ax = b$, where A is symmetric positive definite.

`order`	in	ordering method to use (`0` or `1`)
`A`	in	sparse matrix; only upper triangular part used
`b`	in/out	size `n`; b on input, x on output
	returns	`1` if successful; `0` on error

`cs_compress`: triplet form to compressed-column conversion

```
cs *cs_compress (const cs *T) ;
```

Converts a triplet-form matrix `T` into a compressed-column matrix `C`. The columns of `C` are not sorted, and duplicate entries may be present in `C`.

`T`	in	sparse matrix in triplet form
	returns	`C` if successful; `NULL` on error

`cs_dupl`: remove duplicate entries

```
int cs_dupl (cs *A) ;
```

Removes and sums duplicate entries in a sparse matrix.

`A`	in/out	sparse matrix; duplicates summed on output
	returns	`1` if successful; `0` on error

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

cs_entry: add an entry to a triplet-form matrix

```
int cs_entry (cs *T, int i, int j, double x) ;
```

Memory-space and dimension of `T` are increased if necessary.

`T`	in/out	triplet matrix; new entry added on output
`i`	in	row index of new entry
`j`	in	column index of new entry
`x`	in	numerical value of new entry
	returns	`1` if successful; `0` on error

cs_gaxpy: $y = Ax + y$

```
int cs_gaxpy (const cs *A, const double *x, double *y) ;
```

Sparse matrix times dense column vector, $y = Ax + y$.

`A`	in	sparse matrix
`x`	in	size `n`
`y`	in/out	size `m`
	returns	`1` if successful; `0` on error

cs_load: load a matrix from a file

```
cs *cs_load (FILE *f) ;
```

Loads a triplet matrix `T` from a file. Each line of the file contains three values: a row index i, a column index j, and a numerical value a_{ij}. The file is zero-based.

`f`	in	file pointer to an open file
	returns	`T` if successful; `0` on error

cs_lusol: solve $Ax = b$ using LU factorization

```
int cs_lusol (int order, const cs *A, double *b, double tol) ;
```

Solves $Ax = b$, where A is square and nonsingular. The diagonal entry is selected if $a_{kk}^{[k-1]} \geq$ `tol` $\times \max |a_{*k}^{[k-1]}|$. Partial pivoting if `tol=1`.

`order`	in	ordering method to use (`0` to `3`)
`A`	in	sparse matrix
`b`	in/out	size `n`; b on input, x on output
`tol`	in	partial pivoting tolerance
	returns	`1` if successful; `0` on error

cs_multiply: $C = AB$

```
cs *cs_multiply (const cs *A, const cs *B) ;
```

Sparse matrix multiplication, $C = AB$.

`A`	in	sparse matrix
`B`	in	sparse matrix
	returns	`C=A*B`; `NULL` on error

cs_norm: matrix 1-norm

```
double cs_norm (const cs *A) ;
```

Computes the 1-norm of a sparse matrix.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

`A`	in	sparse matrix
	returns	the 1-norm if successful; `-1` on error

`cs_print`: print a sparse matrix

```
int cs_print (const cs *A, int brief) ;
```

Prints a compressed-column or triplet-form sparse matrix.

`A`	in	sparse matrix
`brief`	in	print all of `A` if zero, a few entries otherwise
	returns	`1` if successful; `0` on error

`cs_qrsol`: solve a least squares or underdetermined problem

```
int cs_qrsol (int order, const cs *A, double *b) ;
```

Solves a least squares problem ($\min \|Ax-b\|_2$, where A is m-by-n with $m \geq n$), or an underdetermined system ($Ax = b$, where $m < n$).

`order`	in	ordering method to use (`0` or `3`)
`A`	in	sparse matrix
`b`	in/out	size `max(m,n)`; b (size `m`) on input, x (size `n`) on output
	returns	`1` if successful; `0` on error

`cs_transpose`: $C = A^T$

```
cs *cs_transpose (const cs *A, int values) ;
```

`A`	in	sparse matrix
`values`	in	pattern only if `0`, both pattern and values otherwise
	returns	`C=A'`; `NULL` on error

Primary CSparse utilities

`cs_calloc`: allocate and clear memory

```
void *cs_calloc (int n, size_t size) ;
```

`n`	in	number of items to allocate
`size`	in	size of each item in bytes
	returns	pointer to allocated block if successful; `NULL` on error

`cs_free`: free memory

```
void *cs_free (void *p) ;
```

`p`	in/out	block to free
	returns	`NULL`

`cs_realloc`: change size of a block of memory

```
void *cs_realloc (void *p, int n, size_t size, int *ok) ;
```

`p`	in/out	block to change
`n`	in	new size of the block in number of items
`size`	in	size of each item, in bytes
`ok`	out	`1` if successful; `0` on error
	returns	pointer to possibly moved block of memory

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

cs_spalloc: allocate a sparse matrix

```
cs *cs_spalloc (int m, int n, int nzmax, int values, int triplet) ;
```

Allocates a sparse matrix in either compressed-column or triplet form.

`m`	in	number of rows
`n`	in	number of columns
`nzmax`	in	maximum number of entries
`values`	in	allocate pattern only if `0`, values and pattern otherwise
`triplet`	in	compressed-column if `0`, triplet form otherwise
	returns	`A` if successful; `NULL` on error

cs_spfree: free a sparse matrix

```
cs *cs_spfree (cs *A) ;
```

Frees a sparse matrix, in either compressed-column or triplet form.

`A`	in/out	sparse matrix to free
	returns	`NULL`

cs_sprealloc: reallocate a sparse matrix

```
int cs_sprealloc (cs *A, int nzmax) ;
```

Changes the maximum number of entries a sparse matrix can hold.

`A`	in/out	matrix to reallocate (compressed-column or triplet)
`nzmax`	in	new maximum number of entries
	returns	`1` if successful; `0` on error

cs_malloc: allocate memory

```
void *cs_malloc (int n, size_t size) ;
```

Allocates an uninitialized block of memory of size `n` items.

`n`	in	number of items to allocate
`size`	in	size of each item in bytes
	returns	pointer to allocated block if successful; `NULL` on error

9.2 Secondary CSparse routines and definitions

css: symbolic analysis

```
typedef struct cs_symbolic  /* symbolic Cholesky, LU, or QR analysis */
{
    int *pinv ;     /* inverse row perm. for QR, fill red. perm for Chol */
    int *q ;        /* fill-reducing column permutation for LU and QR */
    int *parent ;   /* elimination tree for Cholesky and QR */
    int *cp ;       /* column pointers for Cholesky, row counts for QR */
    int *leftmost ; /* leftmost[i] = min(find(A(i,:))), for QR */
    int m2 ;        /* # of rows for QR, after adding fictitious rows */
    double lnz ;    /* # entries in L for LU or Cholesky; in V for QR */
    double unz ;    /* # entries in U for LU; in R for QR */
} css ;
```

csn: numeric factorization

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
typedef struct cs_numeric   /* numeric Cholesky, LU, or QR factorization */
{
    cs *L ;         /* L for LU and Cholesky, V for QR */
    cs *U ;         /* U for LU, R for QR, not used for Cholesky */
    int *pinv ;     /* partial pivoting for LU */
    double *B ;     /* beta [0..n-1] for QR */
} csn ;
```

csd: Dulmage–Mendelsohn decomposition

```
typedef struct cs_dmperm_results    /* cs_dmperm or cs_scc output */
{
    int *p ;        /* size m, row permutation */
    int *q ;        /* size n, column permutation */
    int *r ;        /* size nb+1, block k is rows r[k] to r[k+1]-1 in A(p,q) */
    int *s ;        /* size nb+1, block k is cols s[k] to s[k+1]-1 in A(p,q) */
    int nb ;        /* # of blocks in fine dmperm decomposition */
    int rr [5] ;    /* coarse row decomposition */
    int cc [5] ;    /* coarse column decomposition */
} csd ;
```

cs_amd: approximate minimum degree ordering

```
int *cs_amd (int order, const cs *A) ;
```

Minimum degree ordering of $A + A^T$ or $A^T A$.

`order`	in	ordering method to use (`0` to `3`)
`A`	in	matrix to order
	returns	size `n` permutation; `NULL` on error or for natural ordering

cs_chol: Cholesky factorization

```
csn *cs_chol (const cs *A, const css *S) ;
```

Cholesky factorization $LL = PAP^T$.

`A`	in	matrix to factorize, only upper triangular part used
`S`	in	symbolic analysis from `cs_schol`
	returns	numeric factorization `N`; `NULL` on error

cs_dmperm: Dulmage–Mendelsohn decomposition

```
csd *cs_dmperm (const cs *A, int seed) ;
```

Dulmage–Mendelsohn decomposition. `seed` optionally selects a randomized algorithm.

`A`	in	matrix to order
`seed`	in	`0`: natural, `-1`: reverse, random order otherwise
	returns	Dulmage–Mendelsohn analysis `D`; `NULL` on error

cs_droptol: drop small entries

```
int cs_droptol (cs *A, double tol) ;
```

Removes entries from a matrix with absolute value $\leq$ `tol`.

`A`	in/out	matrix to remove entries from
`tol`	in	drop tolerance
	returns	`1` if successful; `0` on error

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

cs_dropzeros: drop exact zeros

```
int cs_dropzeros (cs *A) ;
```

Removes numerically zero entries from a matrix.

A	in/out	matrix to remove entries from
	returns	1 if successful; 0 on error

cs_happly: apply a Householder reflection

```
int cs_happly (const cs *V, int i, double beta, double *x) ;
```

Applies a Householder reflection to a dense vector, $x = (I - \beta vv^T)x$.

V	in	matrix of Householder vectors
i	in	$v =$ V(:,i), the ith column of V
beta	in	the scalar β
x	in/out	the vector x of size m
	returns	1 if successful; 0 on error

cs_ipvec: $x = P^T b$

```
int cs_ipvec (const int *p, const double *b, double *x, int n) ;
```

Permutes a vector; $x = P^T b$. In MATLAB notation, x(p)=b.

p	in	permutation vector
b	in	input vector
x	out	x(p)=b, output vector
n	in	length of p, b, and x
	returns	1 if successful; 0 on error

cs_lsolve: solve a lower triangular system $Lx = b$

```
int cs_lsolve (const cs *L, double *x) ;
```

Solves a lower triangular system $Lx = b$, where x and b are dense vectors. The diagonal of L must be the first entry of each column.

L	in	lower triangular matrix
x	in/out	size n; right-hand side on input, solution on output
	returns	1 if successful; 0 on error

cs_ltsolve: solve an upper triangular system $L^T x = b$

```
int cs_ltsolve (const cs *L, double *x) ;
```

Same as cs_lsolve, except it solves $L^T x = b$ instead.

cs_lu: sparse LU factorization

```
csn *cs_lu (const cs *A, const css *S, double tol) ;
```

Sparse LU factorization of a square matrix, $PAQ = LU$.

A	in	matrix to factorize
S	in	symbolic analysis from cs_sqr
tol	in	partial pivoting threshold (1 for partial pivoting)
	returns	numeric factorization N; NULL on error

cs_permute: $C = PAQ$

```
cs *cs_permute (const cs *A, const int *pinv, const int *q, int values) ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Permutes a sparse matrix; $C = PAQ$, or `C=A(p,q)` in MATLAB notation.

`A`	in	`m`-by-`n` matrix to permute
`pinv`	in	an inverse permutation vector of length `m`
`q`	in	a permutation vector of length `n`
`values`	in	allocate pattern only if `0`, values and pattern otherwise
	returns	`C=A(p,q)`; `NULL` on error

`cs_pinv`: invert a permutation vector

```
int *cs_pinv (const int *p, int n) ;
```

Inverts a permutation vector. Returns `pinv[i]=k` if `p[k]=i` on input.

`p`	in	a permutation vector of length `n`
`n`	in	length of `p`
	returns	`pinv`, an integer vector of length `n`; `NULL` on error

`cs_pvec`: $x = Pb$

```
int cs_pvec (const int *p, const double *b, double *x, int n) ;
```

Permutes a vector; $x = Pb$. In MATLAB notation, `x=b(p)`.

`p`	in	permutation vector
`b`	in	input vector
`x`	out	`x=b(p)`, output vector
`n`	in	length of `p`, `b`, and `x`
	returns	`1` if successful; `0` on error

`cs_qr`: sparse QR factorization

```
csn *cs_qr (const cs *A, const css *S) ;
```

Sparse QR factorization of an m-by-n matrix A, where $m \geq n$.

`A`	in	matrix to factorize
`S`	in	symbolic analysis from `cs_sqr`
	returns	numeric factorization `N`; `NULL` on error

`cs_schol`: symbolic Cholesky ordering and analysis

```
css *cs_schol (int order, const cs *A) ;
```

Symbolic ordering and analysis for a Cholesky factorization.

`order`	in	ordering option (`0` or `1`)
`A`	in	matrix to factorize
	returns	`S`, symbolic analysis for `cs_chol`; `NULL` on error

`cs_sqr`: symbolic QR or LU ordering and analysis

```
css *cs_sqr (int order, const cs *A, int qr) ;
```

Symbolic ordering and analysis for a QR or LU factorization.

`order`	in	ordering method to use (`0` to `3`)
`A`	in	matrix to factorize
`qr`	in	analyze for QR if nonzero or LU if zero
	returns	`S`, symbolic analysis for `cs_qr` or `cs_lu`; `NULL` on error

`cs_symperm`: $C = PAP^T$ for a symmetric matrix A

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
cs *cs_symperm (const cs *A, const int *pinv, int values) ;
```

Symmetric permutation of a symmetric matrix A.

`A`	in	matrix to permute (only upper triangular part used)
`pinv`	in	size `n`; inverse permutation
`values`	in	allocate pattern only if `0`, values and pattern otherwise
	returns	`C=A(p,p)`; `NULL` on error

`cs_updown`: rank-1 update/downdate $LL^T \pm cc^T$

```
int cs_updown (cs *L, int sigma, const cs *C, const int *parent) ;
```

Sparse Cholesky rank-1 update/downdate. The nonzero pattern $\mathcal{C}$ of c must be a subset of the nonzero pattern of column k of L, where $k = \min \mathcal{C}$.

`L`	in/out	factorization to update/downdate
`sigma`	in	`+1` for update, `-1` for downdate
`C`	in	the vector c
`parent`	in	the elimination tree of `L`
	returns	`1` if successful; `0` on error

`cs_usolve`: solve an upper triangular system $Ux = b$

```
int cs_usolve (const cs *U, double *x) ;
```

Solves an upper triangular system $Ux = b$, where x and b are dense vectors. The diagonal of `U` must be the last entry of each column.

`U`	in	upper triangular matrix
`x`	in/out	size `n`; right-hand side on input, solution on output
	returns	`1` if successful; `0` on error

`cs_utsolve`: solve a lower triangular system $U'x = b$

```
int cs_utsolve (const cs *U, double *x) ;
```

Same as `cs_usolve`, except it solves $U^T x = b$ instead.

Secondary CSparse utilities

`cs_sfree`: free a `css` symbolic analysis

```
css *cs_sfree (css *S) ;
```

`S`	in/out	symbolic analysis to free
	returns	`NULL`

`cs_nfree`: free a `csn` numeric factorization

```
csn *cs_nfree (csn *N) ;
```

`N`	in/out	numeric factorization to free
	returns	`NULL`

`cs_dfree`: free a `csd` Dulmage–Mendelsohn decomposition

```
csd *cs_dfree (csd *D) ;
```

`D`	in/out	decomposition to free
	returns	`NULL`

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

9.3 Tertiary CSparse routines and definitions

cs_counts: column counts for Cholesky factorization of A or A^TA

```
int *cs_counts (const cs *A, const int *parent, const int *post, int ata) ;
```

A	in	matrix to analyze
parent	in	elimination tree of A
post	in	postordering of parent
ata	in	analyze A if zero, A^TA otherwise
	returns	c, a vector of length n; NULL on error

cs_cumsum: cumulative sum of an integer vector

```
double cs_cumsum (int *p, int *c, int n) ;
```

p	out	size n+1; cumulative sum of c
c	in/out	size n; overwritten with p[0..n-1] on output
n	in	length of c; length of p is n+1
	returns	sum(c); -1 on error

cs_dfs: depth-first search of a directed graph

```
int cs_dfs (int j, cs *G, int top, int *xi, int *pstack, const int *pinv) ;
```

j	in	starting node
G	in	graph to search (G->p modified, then restored)
top	in	stack xi[top ... n-1] in use on input
xi	in/out	size n; stack containing nodes traversed
pstack	work	size n
pinv	in	mapping of rows to columns of G, ignored if NULL
	returns	new value of top; -1 on error

cs_ereach: nonzero pattern of kth row of Cholesky factor, L(k,1:k-1)

```
int cs_ereach (const cs *A, int k, const int *parent, int *s, int *w) ;
```

A	in	L is the Cholesky factor of A
k	in	find kth row of L
parent	in	elimination tree of A
s	out	size n; s[top...n-1] is nonzero pattern of L(k,1:k-1)
w	work	size n; w[0...n-1]>=0 on input, unchanged on output
	returns	top; -1 on error

cs_etree: elimination tree of A or A^TA

```
int *cs_etree (const cs *A, int ata) ;
```

A	in	matrix to analyze
ata	in	analyze A if zero, A^TA otherwise
	returns	parent of size n; NULL on error

cs_fkeep: drop entries from a sparse matrix

```
int cs_fkeep (cs *A, int (*fkeep) (int, int, double, void *), void *other) ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

`A`	in/out	matrix to remove entries from
`fkeep`	in	drop a_{ij} if `fkeep(i,j,aij,other)` is zero.
`other`	any	optional parameter to `fkeep`
	returns	`nz`, new number of entries in `A`; `-1` on error

`cs_house`: compute a Householder reflection

```
double cs_house (double *x, double *beta, int n) ;
```

Computes a Householder reflection $H = I - \beta vv^T$ so that $(Hx)_{2\ldots n} = 0$.

`x`	in/out	x on input, v on output
`beta`	out	the scalar β
`n`	in	the length of `x`
	returns	$\|x\|_2$; `-1` on error

`cs_leaf`: determine if j is a leaf and find least common ancestor

Determine if j is a leaf of $\mathcal{T}^i$ and return `q` $= lca(j_{\text{prev}}, j)$. See page 51.

`cs_maxtrans`: maximum matching

```
int *cs_maxtrans (const cs *A, int seed) ;
```

Finds a zero-free diagonal. `seed` optionally selects a randomized algorithm.

`A`	in	matrix to find matching for
`seed`	in	`0`: natural, `-1`: reverse, randomized otherwise
	returns	`jimatch`, row and column matching, size `m+n`

`cs_post`: postorder a tree or forest

```
int *cs_post (const int *parent, int n) ;
```

`parent`	in	defines a tree of `n` nodes
`n`	in	length of `parent`
	returns	`post[k]=i`, of size `n`; `NULL` on error

`cs_randperm`: random permutation

```
int *cs_randperm (int n, int seed) ;
```

`n`	in	length of `p`
`seed`	in	`0`: natural, `-1`: reverse, random `p` otherwise
	returns	random perm. `p`; `NULL` on error or for natural order

`cs_reach`: nonzero pattern of sparse triangular solve

```
int cs_reach (cs *G, const cs *B, int k, int *xi, const int *pinv) ;
```

`G`	in	graph to search (`G->p` modified, then restored)
`B`	in	right-hand side, $b = B(:,k)$
`k`	in	use `k`th column of `B`
`xi`	out	size `2*n`; output in `xi[top...n-1]`
`pinv`	in	mapping of rows to columns of `G`, ignored if `NULL`
	returns	`top`; `-1` on error

`cs_scatter`: scatter a sparse vector

```
int cs_scatter (const cs *A, int j, double beta, int *w, double *x, int mark,
    cs *C, int nz) ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Scatters and sums a sparse vector `A(:,j)` into a dense vector, `x=x+beta*A(:,j)`.

`A`	in	the sparse vector is `A(:,j)`
`j`	in	the column of `A` to use
`beta`	in	scalar multiplied by `A(:,j)`
`w`	in/out	size `m`; node `i` is marked if `w[i]=mark`
`x`	in/out	size `m`; ignored if `NULL`
`mark`	in	mark value for `w`
`C`	in/out	pattern of `x` accumulated in `C->i`
`nz`	in	pattern of `x` placed in `C` starting at `C->i [nz]`
	returns	new value of `nz`; `-1` on error

`cs_scc`: strongly connected components of a square matrix

```
csd *cs_scc (cs *A) ;
```

`A`	in	matrix to analyze (`A->p` modified then restored)
	returns	strongly connected components `D`; `NULL` on error

`cs_spsolve`: sparse lower or upper triangular solve, $Lx = b$ or $Ux = b$

```
int cs_spsolve (cs *G, const cs *B, int k, int *xi, double *x,
    const int *pinv, int lo) ;
```

If `lo` is zero, $Ux = B_{*k}$ is solved, where $G = U$ is upper triangular. Otherwise $Lx = B_{*k}$ is solved instead, where $G = L$. Both $b = B_{*k}$ and x are sparse; $\mathcal{X}$ is the nonzero pattern of x.

`G`	in	lower or upper triangular matrix (L or U)
`B`	in	right-hand side, $b = B_{*k}$
`k`	in	use `k`th column of `B` as right-hand side
`xi`	out	size `2*n`; $\mathcal{X}$ in `xi[top...n-1]`
`x`	out	size `n`; x in `x[xi[top...n-1]]`
`pinv`	in	mapping of rows to columns of `L`, ignored if `NULL`
`lo`	in	`1` if lower triangular, `0` if upper
	returns	`top`; `-1` on error

`cs_tdfs`: postorder a tree

```
int cs_tdfs (int j, int k, int *head, const int *next, int *post,
    int *stack) ;
```

All arrays are of size `n`, where `n` is the number of nodes in the tree.

`j`	in	postorder the tree rooted at node `j`
`k`	in	number of nodes ordered so far
`head`	in/out	`head[i]` is first child of node `i`; `-1` on output
`next`	in	`next[i]` is next sibling of `i` or `-1` if none
`post`	in/out	postordering
`stack`	work	size `n`
	returns	new value of `k`; `-1` on error

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Tertiary CSparse utilities

cs_dalloc: allocate a csd object

```
csd *cs_dalloc (int m, int n) ;
```

Allocates a `csd` object (a Dulmage–Mendelsohn decomposition).

`m`	in	number of rows of the matrix A to be analyzed
`n`	in	number of columns of A
	returns	Dulmage–Mendelsohn decomposition `D`; `NULL` on error

cs_ddone: return a csd Dulmage–Mendelsohn decomposition

```
csd *cs_ddone (csd *D, cs *C, void *w, int ok) ;
```

Frees internally allocated workspace and returns a `csd` result or `NULL` if an error has occurred. The result is freed if `ok` is zero.

`D`	in/out	`csd` result
`C`	in/out	temporary sparse matrix to free
`w`	in/out	workspace to free
`ok`	in	free `D` if zero, keep `D` otherwise
	returns	`D`; `NULL` on error

cs_done: free workspace and return a sparse matrix result

```
cs *cs_done (cs *C, void *w, void *x, int ok) ;
```

Frees internally allocated workspace and returns a sparse matrix result or `NULL` if an error has occurred. The result is freed if `ok` is zero.

`C`	in/out	sparse matrix result
`w`	in/out	workspace to free
`x`	in/out	workspace to free
`ok`	in	free `C` if zero, keep `C` otherwise
	returns	`C`; `NULL` on error

cs_idone: free workspace and return an int array result

```
int *cs_idone (int *p, cs *C, void *w, int ok) ;
```

Frees internally allocated workspace and returns an `int` array result or `NULL` if an error has occurred. The result is freed if `ok` is zero.

`p`	in/out	`int` array result
`C`	in/out	temporary sparse matrix to free
`w`	in/out	workspace to free
`ok`	in	free `p` if zero, keep `p` otherwise
	returns	`p`; `NULL` on error

cs_ndone: return a csn numeric factorization

```
csn *cs_ndone (csn *N, cs *C, void *w, void *x, int ok) ;
```

Frees internally allocated workspace and returns a numeric factorization result or `NULL` if an error has occurred. The result is freed if `ok` is zero.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

N	in/out	numeric factorization result
C	in/out	temporary sparse matrix to free
w	in/out	workspace to free
x	in/out	workspace to free
ok	in	free N if zero, keep N otherwise
	returns	N; NULL on error

Macros

The cs.h include file starts with the following lines. The _CS_H definition ensures that cs.h can be included multiple times without causing an error. Four ANSI standard definition files are included (stdlib.h, limits.h, math.h, and stdio.h). The MATLAB mex.h file is included if CSparse is being compiled for MATLAB. Next, the CSparse version numbers, release date, and copyright are defined (for use in cs_print).

```
#ifndef _CS_H
#define _CS_H
#include <stdlib.h>
#include <limits.h>
#include <math.h>
#include <stdio.h>
#ifdef MATLAB_MEX_FILE
#include "mex.h"
#endif
#define CS_VER 2                    /* CSparse Version 2.0.1 */
#define CS_SUBVER 0
#define CS_SUBSUB 1
#define CS_DATE "May 27, 2006"      /* CSparse release date */
#define CS_COPYRIGHT "Copyright (c) Timothy A. Davis, 2006"
```

Each one-line prototype described in this chapter is listed in cs.h. The cs.h file ends with the following lines, which define the CSparse macros.

```
#define CS_MAX(a,b) (((a) > (b)) ? (a) : (b))
#define CS_MIN(a,b) (((a) < (b)) ? (a) : (b))
#define CS_FLIP(i) (-(i)-2)
#define CS_UNFLIP(i) (((i) < 0) ? CS_FLIP(i) : (i))
#define CS_MARKED(w,j) (w [j] < 0)
#define CS_MARK(w,j) { w [j] = CS_FLIP (w [j]) ; }
#define CS_CSC(A) (A && (A->nz == -1))
#define CS_TRIPLET(A) (A && (A->nz >= 0))
#endif
```

9.4 Examples

The three example programs below exercise every routine and nearly every line of code in CSparse (all but out-of-memory condition handling).

cs_demo1: file I/O and basic matrix operations

The complete program cs_demo1 reads in a triplet matrix from stdin and performs various matrix operations.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
#include "cs.h"
int main (void)
{
    cs *T, *A, *Eye, *AT, *C, *D ;
    int i, m ;
    T = cs_load (stdin) ;               /* load triplet matrix T from stdin */
    printf ("T:\n") ; cs_print (T, 0) ; /* print T */
    A = cs_compress (T) ;               /* A = compressed-column form of T */
    printf ("A:\n") ; cs_print (A, 0) ; /* print A */
    cs_spfree (T) ;                     /* clear T */
    AT = cs_transpose (A, 1) ;          /* AT = A' */
    printf ("AT:\n") ; cs_print (AT, 0) ; /* print AT */
    m = A ? A->m : 0 ;                  /* m = # of rows of A */
    T = cs_spalloc (m, m, m, 1, 1) ;    /* create triplet identity matrix */
    for (i = 0 ; i < m ; i++) cs_entry (T, i, i, 1) ;
    Eye = cs_compress (T) ;             /* Eye = speye (m) */
    cs_spfree (T) ;
    C = cs_multiply (A, AT) ;           /* C = A*A' */
    D = cs_add (C, Eye, 1, cs_norm (C)) ;   /* D = C + Eye*norm (C,1) */
    printf ("D:\n") ; cs_print (D, 0) ; /* print D */
    cs_spfree (A) ;                     /* clear A AT C D Eye */
    cs_spfree (AT) ;
    cs_spfree (C) ;
    cs_spfree (D) ;
    cs_spfree (Eye) ;
    return (0) ;
}
```

The `t1` file can be used as input to `cs_demo1`. It contains the triplet form of the matrix used in Section 2.1:

```
2 2 3.0
1 0 3.1
3 3 1.0
0 2 3.2
1 1 2.9
3 0 3.5
3 1 0.4
1 3 0.9
0 0 4.5
2 1 1.7
```

The `cs_demo1.m` script below is the MATLAB equivalent for the C program `cs_demo1`, except that the CSparse results are compared with the same operations in MATLAB. The MATLAB `load` statement can read a triplet-form matrix in the same format as the `t1` file above, except that MATLAB expects its matrices to be 1-based. MATLAB always returns matrices with sorted columns.

```
%CS_DEMO1: MATLAB version of the CSparse/Demo/cs_demo1.c program.
% Uses both MATLAB functions and CSparse mexFunctions, and compares the two
% results.  This demo also plots the results, which the C version does not do.

load ../../Matrix/t1
T = t1
A  = sparse    (T(:,1)+1, T(:,2)+1, T(:,3))
A2 = cs_sparse (T(:,1)+1, T(:,2)+1, T(:,3))
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
fprintf ('A difference: %g\n', norm (A-A2,1)) ;
% CSparse/Demo/cs_demo1.c also clears the triplet matrix T at this point:
% clear T
clf
subplot (2,2,1) ; cspy (A) ; title ('A', 'FontSize', 16) ;
AT = A'
AT2 = cs_transpose (A)
fprintf ('AT difference: %g\n', norm (AT-AT2,1)) ;
subplot (2,2,2) ; cspy (AT) ; title ('A''', 'FontSize', 16) ;
n = size (A,2) ;
I = speye (n) ;
C = A*AT ;
C2 = cs_multiply (A, AT)
fprintf ('C difference: %g\n', norm (C-C2,1)) ;
subplot (2,2,3) ; cspy (C) ; title ('C=A*A''', 'FontSize', 16) ;
cnorm = norm (C,1) ;
D = C + I*cnorm
D2 = cs_add (C, I, 1, cnorm)
fprintf ('D difference: %g\n', norm (D-D2,1)) ;
subplot (2,2,4) ; cspy (D) ; title ('D=C+I*norm(C,1)', 'FontSize', 16) ;
% CSparse/Demo/cs_demo1.c clears all matrices at this point:
% clear A AT C D I
% clear A2 AT2 C2 D2
```

The output of the C program `cs_demo1` is given below. Compare it with the triplet and compressed-column matrices defined in Section 2.1. Also compare it with the output of the MATLAB equivalent code above. The maximum number of entries that `T` can hold is 16; it was doubled four times from its original size of one entry.

```
T:
CSparse Version 2.0.1, May 27, 2006.  Copyright (c) Timothy A. Davis, 2006
triplet: 4-by-4, nzmax: 16 nnz: 10
    2 2 : 3
    1 0 : 3.1
    3 3 : 1
    0 2 : 3.2
    1 1 : 2.9
    3 0 : 3.5
    3 1 : 0.4
    1 3 : 0.9
    0 0 : 4.5
    2 1 : 1.7
A:
CSparse Version 2.0.1, May 27, 2006.  Copyright (c) Timothy A. Davis, 2006
4-by-4, nzmax: 10 nnz: 10, 1-norm: 11.1
    col 0 : locations 0 to 2
      1 : 3.1
      3 : 3.5
      0 : 4.5
    col 1 : locations 3 to 5
      1 : 2.9
      3 : 0.4
      2 : 1.7
    col 2 : locations 6 to 7
      2 : 3
      0 : 3.2
    col 3 : locations 8 to 9
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
      3 : 1
      1 : 0.9
AT:
CSparse Version 2.0.1, May 27, 2006.  Copyright (c) Timothy A. Davis, 2006
4-by-4, nzmax: 10 nnz: 10, 1-norm: 7.7
    col 0 : locations 0 to 1
      0 : 4.5
      2 : 3.2
    col 1 : locations 2 to 4
      0 : 3.1
      1 : 2.9
      3 : 0.9
    col 2 : locations 5 to 6
      1 : 1.7
      2 : 3
    col 3 : locations 7 to 9
      0 : 3.5
      1 : 0.4
      3 : 1
D:
CSparse Version 2.0.1, May 27, 2006.  Copyright (c) Timothy A. Davis, 2006
4-by-4, nzmax: 16 nnz: 16, 1-norm: 139.58
    col 0 : locations 0 to 3
      1 : 13.95
      3 : 15.75
      0 : 100.28
      2 : 9.6
    col 1 : locations 4 to 7
      1 : 88.62
      3 : 12.91
      0 : 13.95
      2 : 4.93
    col 2 : locations 8 to 11
      1 : 4.93
      3 : 0.68
      2 : 81.68
      0 : 9.6
    col 3 : locations 12 to 15
      1 : 12.91
      3 : 83.2
      0 : 15.75
      2 : 0.68
```

`cs_demo2`: matrix factorization

The `cs_demo2` program reads a matrix from a file in triplet form. If it is upper (or lower) triangular, the matrix is assumed to be symmetric, and the lower (or upper) triangular part is added. The matrix is analyzed with `cs_dmperm` and the number of blocks, number of singletons (1-by-1 blocks), and structural rank are printed. If the matrix is symmetric, a Cholesky factorization is used to solve $Ax = b$. If the matrix is square, the system is also solved using an LU factorization. Next, a least squares problem is solved using QR factorization. Each method is tested with a range of ordering options, and the relative residual $\|Ax - b\|/(\|A\|\|x\| + \|b\|)$ and run time are printed. The `cs_demo.h` file contains prototypes for the `cs_demo2` and

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

cs_demo3 programs.

```
#include "cs.h"
typedef struct problem_struct
{
    cs *A ;
    cs *C ;
    int sym ;
    double *x ;
    double *b ;
    double *resid ;
} problem ;

problem *get_problem (FILE *f, double tol) ;
int demo2 (problem *Prob) ;
int demo3 (problem *Prob) ;
problem *free_problem (problem *Prob) ;
```

The cs_demo.c file contains functions used by several demo programs.

```
#include "cs_demo.h"
#include <time.h>
/* 1 if A is square & upper tri., -1 if square & lower tri., 0 otherwise */
static int is_sym (cs *A)
{
    int is_upper, is_lower, j, p, n = A->n, m = A->m, *Ap = A->p, *Ai = A->i ;
    if (m != n) return (0) ;
    is_upper = 1 ;
    is_lower = 1 ;
    for (j = 0 ; j < n ; j++)
    {
        for (p = Ap [j] ; p < Ap [j+1] ; p++)
        {
            if (Ai [p] > j) is_upper = 0 ;
            if (Ai [p] < j) is_lower = 0 ;
        }
    }
    return (is_upper ? 1 : (is_lower ? -1 : 0)) ;
}

/* true for off-diagonal entries */
static int dropdiag (int i, int j, double aij, void *other) { return (i != j) ;}

/* C = A + triu(A,1)' */
static cs *make_sym (cs *A)
{
    cs *AT, *C ;
    AT = cs_transpose (A, 1) ;          /* AT = A' */
    cs_fkeep (AT, &dropdiag, NULL) ;    /* drop diagonal entries from AT */
    C = cs_add (A, AT, 1, 1) ;          /* C = A+AT */
    cs_spfree (AT) ;
    return (C) ;
}

/* create a right-hand side */
static void rhs (double *x, double *b, int m)
{
    int i ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    for (i = 0 ; i < m ; i++) b [i] = 1 + ((double) i) / m ;
    for (i = 0 ; i < m ; i++) x [i] = b [i] ;
}

/* infinity-norm of x */
static double norm (double *x, int n)
{
    int i ;
    double normx = 0 ;
    for (i = 0 ; i < n ; i++) normx = CS_MAX (normx, fabs (x [i])) ;
    return (normx) ;
}

/* compute residual, norm(A*x-b,inf) / (norm(A,1)*norm(x,inf) + norm(b,inf)) */
static void print_resid (int ok, cs *A, double *x, double *b, double *resid)
{
    int i, m, n ;
    if (!ok) { printf ("    (failed)\n") ; return ; }
    m = A->m ; n = A->n ;
    for (i = 0 ; i < m ; i++) resid [i] = -b [i] ;  /* resid = -b */
    cs_gaxpy (A, x, resid) ;                        /* resid = resid + A*x  */
    printf ("resid: %8.2e\n", norm (resid,m) / ((n == 0) ? 1 :
        (cs_norm (A) * norm (x,n) + norm (b,m)))) ;
}

static double tic (void) { return (clock () / (double) CLOCKS_PER_SEC) ; }
static double toc (double t) { double s = tic () ; return (CS_MAX (0, s-t)) ; }

static void print_order (int order)
{
    switch (order)
    {
        case 0: printf ("natural    ") ; break ;
        case 1: printf ("amd(A+A')  ") ; break ;
        case 2: printf ("amd(S'*S)  ") ; break ;
        case 3: printf ("amd(A'*A)  ") ; break ;
    }
}

/* read a problem from a file */
problem *get_problem (FILE *f, double tol)
{
    cs *T, *A, *C ;
    int sym, m, n, mn, nz1, nz2 ;
    problem *Prob ;
    Prob = cs_calloc (1, sizeof (problem)) ;
    if (!Prob) return (NULL) ;
    T = cs_load (f) ;                   /* load triplet matrix T from a file */
    Prob->A = A = cs_compress (T) ;     /* A = compressed-column form of T */
    cs_spfree (T) ;                     /* clear T */
    if (!cs_dupl (A)) return (free_problem (Prob)) ; /* sum up duplicates */
    Prob->sym = sym = is_sym (A) ;      /* determine if A is symmetric */
    m = A->m ; n = A->n ;
    mn = CS_MAX (m,n) ;
    nz1 = A->p [n] ;
    cs_dropzeros (A) ;                  /* drop zero entries */
    nz2 = A->p [n] ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    if (tol > 0) cs_droptol (A, tol) ;  /* drop tiny entries (just to test) */
    Prob->C = C = sym ? make_sym (A) : A ;  /* C = A + triu(A,1)', or C=A */
    if (!C) return (free_problem (Prob)) ;
    printf ("\n--- Matrix: %d-by-%d, nnz: %d (sym: %d: nnz %d), norm: %8.2e\n",
            m, n, A->p [n], sym, sym ? C->p [n] : 0, cs_norm (C)) ;
    if (nz1 != nz2) printf ("zero entries dropped: %d\n", nz1 - nz2) ;
    if (nz2 != A->p [n]) printf ("tiny entries dropped: %d\n", nz2 - A->p [n]) ;
    Prob->b = cs_malloc (mn, sizeof (double)) ;
    Prob->x = cs_malloc (mn, sizeof (double)) ;
    Prob->resid = cs_malloc (mn, sizeof (double)) ;
    return ((!Prob->b || !Prob->x || !Prob->resid) ? free_problem (Prob) : Prob) ;
}

/* free a problem */
problem *free_problem (problem *Prob)
{
    if (!Prob) return (NULL) ;
    cs_spfree (Prob->A) ;
    if (Prob->sym) cs_spfree (Prob->C) ;
    cs_free (Prob->b) ;
    cs_free (Prob->x) ;
    cs_free (Prob->resid) ;
    return (cs_free (Prob)) ;
}

/* solve a linear system using Cholesky, LU, and QR, with various orderings */
int demo2 (problem *Prob)
{
    cs *A, *C ;
    double *b, *x, *resid,  t, tol ;
    int k, m, n, ok, order, nb, ns, *r, *s, *rr, sprank ;
    csd *D ;
    if (!Prob) return (0) ;
    A = Prob->A ; C = Prob->C ; b = Prob->b ; x = Prob->x ; resid = Prob->resid;
    m = A->m ; n = A->n ;
    tol = Prob->sym ? 0.001 : 1 ;               /* partial pivoting tolerance */
    D = cs_dmperm (C, 1) ;                      /* randomized dmperm analysis */
    if (!D) return (0) ;
    nb = D->nb ; r = D->r ; s = D->s ; rr = D->rr ;
    sprank = rr [3] ;
    for (ns = 0, k = 0 ; k < nb ; k++)
    {
        ns += ((r [k+1] == r [k]+1) && (s [k+1] == s [k]+1)) ;
    }
    printf ("blocks: %d singletons: %d structural rank: %d\n", nb, ns, sprank) ;
    cs_dfree (D) ;
    for (order = 0 ; order <= 3 ; order += 3)   /* natural and amd(A'*A) */
    {
        if (!order && m > 1000) continue ;
        printf ("QR   ") ;
        print_order (order) ;
        rhs (x, b, m) ;                         /* compute right-hand side */
        t = tic () ;
        ok = cs_qrsol (order, C, x) ;           /* min norm(Ax-b) with QR */
        printf ("time: %8.2f ", toc (t)) ;
        print_resid (ok, C, x, b, resid) ;      /* print residual */
    }
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    if (m != n || sprank < n) return (1) ;      /* return if rect. or singular*/
    for (order = 0 ; order <= 3 ; order++)      /* try all orderings */
    {
        if (!order && m > 1000) continue ;
        printf ("LU   ") ;
        print_order (order) ;
        rhs (x, b, m) ;                         /* compute right-hand side */
        t = tic () ;
        ok = cs_lusol (order, C, x, tol) ;      /* solve Ax=b with LU */
        printf ("time: %8.2f ", toc (t)) ;
        print_resid (ok, C, x, b, resid) ;      /* print residual */
    }
    if (!Prob->sym) return (1) ;
    for (order = 0 ; order <= 1 ; order++)      /* natural and amd(A+A') */
    {
        if (!order && m > 1000) continue ;
        printf ("Chol ") ;
        print_order (order) ;
        rhs (x, b, m) ;                         /* compute right-hand side */
        t = tic () ;
        ok = cs_cholsol (order, C, x) ;         /* solve Ax=b with Cholesky */
        printf ("time: %8.2f ", toc (t)) ;
        print_resid (ok, C, x, b, resid) ;      /* print residual */
    }
    return (1) ;
}
```

The `cs_demo2.c` file contains just the main program itself.

```
#include "cs_demo.h"
/* cs_demo2: read a matrix and solve a linear system */
int main (void)
{
    problem *Prob = get_problem (stdin, 1e-14) ;
    demo2 (Prob) ;
    free_problem (Prob) ;
    return (0) ;
}
```

The output of `cs_demo2` for the `bcsstk01`, `fs_183_1`, `mbeacxc`, `west0067`, and `lp_afiro` matrices is shown below. One matrix (`mbeacxc`) is actually 496-by-496, but `cs_load` returns a matrix of size 492-by-490 as determined by the largest row and column index of nonzero entries in the matrix. The matrix has a numeric and structural rank of 448, which is why the residual is `nan`.

```
--- Matrix: 48-by-48, nnz: 224 (sym: -1: nnz 400), norm: 3.57e+09
blocks: 1 singletons: 0 structural rank: 48
QR   natural    time:     0.00 resid: 2.83e-19
QR   amd(A'*A)  time:     0.00 resid: 5.19e-19
LU   natural    time:     0.00 resid: 2.63e-19
LU   amd(A+A')  time:     0.00 resid: 8.63e-20
LU   amd(S'*S)  time:     0.00 resid: 2.04e-19
LU   amd(A'*A)  time:     0.00 resid: 2.04e-19
Chol natural    time:     0.00 resid: 1.90e-19
Chol amd(A+A')  time:     0.00 resid: 2.01e-19
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
--- Matrix: 183-by-183, nnz: 988 (sym: 0: nnz 0), norm: 1.70e+09
zero entries dropped: 71
tiny entries dropped: 10
blocks: 38 singletons: 37 structural rank: 183
QR   natural    time:     0.01 resid: 1.09e-27
QR   amd(A'*A)  time:     0.00 resid: 5.34e-28
LU   natural    time:     0.00 resid: 3.08e-28
LU   amd(A+A')  time:     0.00 resid: 1.42e-27
LU   amd(S'*S)  time:     0.00 resid: 7.11e-28
LU   amd(A'*A)  time:     0.00 resid: 7.11e-28

--- Matrix: 492-by-490, nnz: 49920 (sym: 0: nnz 0), norm: 9.29e-01
blocks: 10 singletons: 8 structural rank: 448
QR   natural    time:     0.24 resid:      nan
QR   amd(A'*A)  time:     0.28 resid:      nan

--- Matrix: 67-by-67, nnz: 294 (sym: 0: nnz 0), norm: 6.14e+00
blocks: 2 singletons: 1 structural rank: 67
QR   natural    time:     0.00 resid: 3.42e-17
QR   amd(A'*A)  time:     0.00 resid: 1.95e-17
LU   natural    time:     0.00 resid: 3.85e-17
LU   amd(A+A')  time:     0.00 resid: 1.95e-17
LU   amd(S'*S)  time:     0.00 resid: 2.60e-17
LU   amd(A'*A)  time:     0.00 resid: 2.60e-17

--- Matrix: 27-by-51, nnz: 102 (sym: 0: nnz 0), norm: 3.43e+00
blocks: 1 singletons: 0 structural rank: 27
QR   natural    time:     0.00 resid: 9.54e-17
QR   amd(A'*A)  time:     0.00 resid: 1.89e-16
```

cs_demo3: Cholesky update/downdate

The cs_demo3 program demonstrates the use of many of the secondary CSparse routines and utilities. It computes the Cholesky factorization of a matrix and then updates and downdates it. The first two functions are in the cs_demo.c file.

```
/* free workspace for demo3 */
static int done3 (int ok, css *S, csn *N, double *y, cs *W, cs *E, int *p)
{
    cs_sfree (S) ;
    cs_nfree (N) ;
    cs_free (y) ;
    cs_spfree (W) ;
    cs_spfree (E) ;
    cs_free (p) ;
    return (ok) ;
}

/* Cholesky update/downdate */
int demo3 (problem *Prob)
{
    cs *A, *C, *W = NULL, *WW, *WT, *E = NULL, *W2 ;
    int n, k, *Li, *Lp, *Wi, *Wp, p1, p2, *p = NULL, ok ;
    double *b, *x, *resid, *y = NULL, *Lx, *Wx, s,  t, t1 ;
    css *S = NULL ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
csn *N = NULL ;
if (!Prob || !Prob->sym || Prob->A->n == 0) return (0) ;
A = Prob->A ; C = Prob->C ; b = Prob->b ; x = Prob->x ; resid = Prob->resid;
n = A->n ;
if (!Prob->sym || n == 0) return (1) ;
rhs (x, b, n) ;                             /* compute right-hand side */
printf ("\nchol then update/downdate ") ;
print_order (1) ;
y = cs_malloc (n, sizeof (double)) ;
t = tic () ;
S = cs_schol (1, C) ;                       /* symbolic Chol, amd(A+A') */
printf ("\nsymbolic chol time %8.2f\n", toc (t)) ;
t = tic () ;
N = cs_chol (C, S) ;                        /* numeric Cholesky */
printf ("numeric  chol time %8.2f\n", toc (t)) ;
if (!S || !N || !y) return (done3 (0, S, N, y, W, E, p)) ;
t = tic () ;
cs_ipvec (S->pinv, b, y, n) ;               /* y = P*b */
cs_lsolve (N->L, y) ;                       /* y = L\y */
cs_ltsolve (N->L, y) ;                      /* y = L'\y */
cs_pvec (S->pinv, y, x, n) ;                /* x = P'*y */
printf ("solve    chol time %8.2f\n", toc (t)) ;
printf ("original: ") ;
print_resid (1, C, x, b, resid) ;           /* print residual */
k = n/2 ;                                   /* construct W  */
W = cs_spalloc (n, 1, n, 1, 0) ;
if (!W) return (done3 (0, S, N, y, W, E, p)) ;
Lp = N->L->p ; Li = N->L->i ; Lx = N->L->x ;
Wp = W->p ; Wi = W->i ; Wx = W->x ;
Wp [0] = 0 ;
p1 = Lp [k] ;
Wp [1] = Lp [k+1] - p1 ;
s = Lx [p1] ;
srand (1) ;
for ( ; p1 < Lp [k+1] ; p1++)
{
    p2 = p1 - Lp [k] ;
    Wi [p2] = Li [p1] ;
    Wx [p2] = s * rand () / ((double) RAND_MAX) ;
}
t = tic () ;
ok = cs_updown (N->L, +1, W, S->parent) ;   /* update: L*L'+W*W' */
t1 = toc (t) ;
printf ("update:   time: %8.2f\n", t1) ;
if (!ok) return (done3 (0, S, N, y, W, E, p)) ;
t = tic () ;
cs_ipvec (S->pinv, b, y, n) ;               /* y = P*b */
cs_lsolve (N->L, y) ;                       /* y = L\y */
cs_ltsolve (N->L, y) ;                      /* y = L'\y */
cs_pvec (S->pinv, y, x, n) ;                /* x = P'*y */
t = toc (t) ;
p = cs_pinv (S->pinv, n) ;
W2 = cs_permute (W, p, NULL, 1) ;           /* E = C + (P'W)*(P'W)' */
WT = cs_transpose (W2,1) ;
WW = cs_multiply (W2, WT) ;
cs_spfree (WT) ;
cs_spfree (W2) ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    E = cs_add (C, WW, 1, 1) ;
    cs_spfree (WW) ;
    if (!E || !p) return (done3 (0, S, N, y, W, E, p)) ;
    printf ("update:   time: %8.2f (incl solve) ", t1+t) ;
    print_resid (1, E, x, b, resid) ;           /* print residual */
    cs_nfree (N) ;                              /* clear N */
    t = tic () ;
    N = cs_chol (E, S) ;                        /* numeric Cholesky */
    if (!N) return (done3 (0, S, N, y, W, E, p)) ;
    cs_ipvec (S->pinv, b, y, n) ;               /* y = P*b */
    cs_lsolve (N->L, y) ;                       /* y = L\y */
    cs_ltsolve (N->L, y) ;                      /* y = L'\y */
    cs_pvec (S->pinv, y, x, n) ;                /* x = P'*y */
    t = toc (t) ;
    printf ("rechol:   time: %8.2f (incl solve) ", t) ;
    print_resid (1, E, x, b, resid) ;           /* print residual */
    t = tic () ;
    ok = cs_updown (N->L, -1, W, S->parent) ;   /* downdate: L*L'-W*W' */
    t1 = toc (t) ;
    if (!ok) return (done3 (0, S, N, y, W, E, p)) ;
    printf ("downdate: time: %8.2f\n", t1) ;
    t = tic () ;
    cs_ipvec (S->pinv, b, y, n) ;               /* y = P*b */
    cs_lsolve (N->L, y) ;                       /* y = L\y */
    cs_ltsolve (N->L, y) ;                      /* y = L'\y */
    cs_pvec (S->pinv, y, x, n) ;                /* x = P'*y */
    t = toc (t) ;
    printf ("downdate: time: %8.2f (incl solve) ", t1+t) ;
    print_resid (1, C, x, b, resid) ;           /* print residual */
    return (done3 (1, S, N, y, W, E, p)) ;
}
```

The `cs_demo3.c` file contains just the main program itself.

```
#include "cs_demo.h"
/* cs_demo3: read a matrix and test Cholesky update/downdate */
int main (void)
{
    problem *Prob = get_problem (stdin, 0) ;
    demo3 (Prob) ;
    free_problem (Prob) ;
    return (0) ;
}
```

The output of `cs_demo3` for the `bcsstk16` matrix is shown below.

```
--- Matrix: 4884-by-4884, nnz: 147631 (sym: -1: nnz 290378), norm: 7.01e+09

chol then update/downdate amd(A+A')
symbolic chol time     0.03
numeric  chol time     0.54
solve    chol time     0.01
original: resid: 9.28e-23
update:   time:     0.00
update:   time:     0.01 (incl solve) resid: 1.05e-23
rechol:   time:     0.55 (incl solve) resid: 8.72e-24
downdate: time:     0.01
downdate: time:     0.02 (incl solve) resid: 3.60e-22
```